Computer Networks - Assignment 1

UDP Pinger and UDP Heartbeat sensor

Part 1 - UDP Pinger:

Overview:

The Python code in this assignment's part 1 implements a simple UDP-based pinger application designed to measure network latency and packet loss between a client and a server. The system operates on the following principles:

- **UDP Communication:** The client and server use the User Datagram Protocol (UDP) for communication, a connectionless protocol offering lower overhead but lacking reliability guarantees.
- **Ping Requests:** The client sends ping requests to the server, consisting of a sequence number and a timestamp.
- **Server Responses:** The server responds to each ping request with the same message, including the received timestamp.
- **RTT Measurement:** The client calculates the Round-Trip Time (RTT) for each ping request by subtracting the send time from the receive time.
- **Statistics Calculation:** The client gathers statistics such as the minimum, maximum, and average RTT, as well as the packet loss rate.

Code Components:

Client:

- **Socket Creation:** Creates a UDP socket using the socket.socket() function, specifying the address family (AF_INET for IPv4) and socket type (SOCK_DGRAM for UDP).

- **Ping Request Generation:** Generates ping requests by creating a string with a sequence number and the current time.
- **Packet Transmission:** Sends ping requests to the server using the sendto() method of the socket, specifying the message and the server's address.
- **Response Reception:** Receives server responses using the recvfrom() method, which also returns the sender's address.
- **RTT Calculation:** Calculates the RTT by subtracting the send time from the receive time.
- **Statistics Calculation:** Maintains lists to store RTT values and failed packets. It calculates the minimum, maximum, average RTT, and packet loss rate.
- **Timeout Handling:** Uses a try-except block with the exception of handling cases where responses are not received within a specified time(or responses are not received for any other reason, for that matter).

Server:

- **Socket Binding:** Binds the socket to a specific port and IP address using the bind() method.
- **Packet Reception:** Receives ping requests using the recvfrom() method.
- **Response Generation:** Creates a response message by converting the received message to uppercase.
- **Packet Transmission:** Sends responses back to the client using the sendto() method.

Assumptions and Limitations:

- **Local Network:** For simplicity, the client and server are assumed to be on the same local network.
- **UDP Reliability:** UDP is inherently unreliable, and there's no guarantee of packet delivery. The code accounts for this by handling timeouts and calculating packet loss.

- **Server Response Time:** The server's response time is assumed to be negligible compared to network latency.
- **Random Packet Loss:** The server's random packet loss mechanism is for demonstration purposes and may not accurately simulate real-world network conditions.

Part 2 - UDP Heartbeat sensor:

Overview:

The Python code in this assignment's part 2 implements a UDP-based heartbeat mechanism to monitor the availability of a server. The system operates on the following principles:

- **UDP Communication: The client and server use the User Datagram Protocol (UDP) for communication, a connectionless protocol offering lower overhead.**
- **Heartbeat Messages: The client periodically sends heartbeat messages to the server, including a sequence number and timestamp.**
- **Server Responses: The server responds to heartbeat messages, indicating its availability.**
- **Packet Loss Simulation: The server can simulate packet loss to test the client's resilience.**
- **Server Down Detection: The client detects server downtime by counting consecutive failed heartbeat attempts.**

Code Components:

Client:

- **Socket Creation: Creates a UDP socket using the `socket.socket()` function, specifying the address family (AF_INET for IPv4) and socket type (SOCK_DGRAM for UDP).**

- **Heartbeat Message Generation:** Generates heartbeat messages with a sequence number and timestamp.
- **Packet Transmission:** Sends heartbeat messages to the server using the `sendto()` method.
- **Response Reception:** Attempts to receive responses from the server within a timeout.
- **Server Down Detection:** Counts consecutive failed heartbeat attempts and declares the server down if the count exceeds a threshold.

Server:

- **Socket Creation:** Creates a UDP socket using the `socket.socket()` function, specifying the address family (AF_INET for IPv4) and socket type (SOCK_DGRAM for UDP).
- **Socket Binding:** Binds the socket to a specific port and IP address using the `bind()` method.
- **Packet Loss Simulation:** Simulates packet loss based on a user-defined percentage.
- **Heartbeat Message Processing:** Receives heartbeat messages, extracts the sequence number and timestamp, calculates the elapsed time, and sends a response.

Assumptions and Limitations:

- **Local Network:** The client and server are assumed to be on the same local network.
- **UDP Reliability:** UDP is inherently unreliable, and there's no guarantee of packet delivery. The code accounts for this by handling timeouts and detecting server downtime.
- **Packet Loss Simulation:** The simulated packet loss may not accurately reflect real-world network conditions.

- **Server Response Time:** The server's response time is assumed to be negligible compared to network latency.
- **Client Configuration:** The client must be configured with the correct server IP address, port, and maximum allowed losses.